The East Kalyan Welfare Society's
MODEL COLLEGE OF SCIENCE AND COMMERCE
(Affiliated to Mumbai University)
CHINCHPADA ROAD, RAJBHAR NAGAR, KALYAN (E)

INDEX

Sr. No.	Title	Date	Sign
	Module 1		
1.	**Android Basics and Resources**		
	1. Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.	**17/6/26**	
	2. Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.	**17/6/26**	
	3. Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.	**20/6/26**	
2.	**Fragments and Activity Interaction**		
	1. Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.	**24/6/26**	
	2. Develop an Android application implementing Fragment-based UI navigation between multiple screens.	**24/6/26**	
	3. Develop an Android application demonstrating communication between Activity and Fragment.	**1/7/26**	
3.	**Basic Layout Management**		
	1. Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.	**1/7/26**	
	2. Develop an Android application using RelativeLayout to align UI components dynamically.	**8/7/26**	
	3. Develop an Android application using TableLayout to design a structured Login/Registration Form.	**8/7/26**	
4	**Advanced Layout and View Handling**		
	1. Develop an Android application using AbsoluteLayout demonstrating fixed positioning.	**11/7/26**	
	2. Develop an Android application using FrameLayout to overlay UI elements.	**11/7/26**	

	3. Develop an Android application using ScrollView to display long scrollable content.	**15/7/26**	
5	**Adapter Based Views**		
	1. 1. Develop an Android application using ListView with ArrayAdapter and implement item click handling.	**15/7/26**	
	2. 2. Develop an Android application using GridView with Custom Adapter and image selection handling.	**18/7/26**	
	3. 3. Develop an Android application implementing RecyclerView as a modern alternative to ListView.	**18/7/26**	
	Module-2		
1	**Menus, Dialogs and UI Interaction**		
	1. Develop an Android application implementing AppBar (Toolbar) and Options Menu.	**22/7/26**	
	2. 2. Develop an Android application demonstrating AlertDialog and custom DialogFragment.	**22/7/26**	
	3. 3. Develop an Android application implementing contextual menus and popup menus.	**25/7/26**	
2	**Intents, Events and Listeners**		
	1. Develop an Android application implementing Explicit and Implicit Intents.	**25/7/26**	
	2. Develop an Android application demonstrating Event Handling using Listeners	**5/8/16**	
	3. Develop an Android application using Adapters to populate Spinner or ListView	**8/8/26**	
3	**Services, Notifications and Broadcast Receivers**		
	1. Develop an Android application implementing Started and Bound Services.	**12/8/26**	
	2. Develop an Android application implementing Notifications using Notification Manager.	**22/8/26**	
	3. Develop an Android application implementing Static and Dynamic Broadcast Receivers.	**25/8/26**	
4	**Database and Networking**		
	1. Develop an Android application implementing SQLite database CRUD operations.	**7/9/26**	
	2. Develop an Android application to fetch and parse JSON data from a web service and display it in UI components.	**8/9/26**	
5	**Concurrency, Media and Security**		
	1. Develop an Android application demonstrating Threads and Handlers for background processing.	**12/9/26**	
	2. Develop an Android application implementing Media APIs and Telephone APIs.	**21/9/26**	
	3. Develop an Android application implementing Runtime Permissions.	**24/9/26**	

Module 1

Practical No. 1

Android Basics and Resources

1. Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.
2. Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.
3. Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

1. Aim : To print hello world

MainActivity.java

```
package com.example.helloworld;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;


public class MainActivity extends AppCompatActivity {


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);  // Loads the UI
    }
}
```

activity_main.xml

(Located in: app → res → layout → activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">


    <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    android:textSize="30sp" />
</LinearLayout>
```

2. Run the App

- Click Run ▶


Pixel 9A API 34
10:25
HelloWorld
Hello World!

2.Android Resources: Colors, Themes, Strings, Drawables, Dimensions, Images

AIM: To study and implement Android Resources such as Color, Theme, String, Drawable, Dimension, and Image in an Android application.

THEORY:

Android resources are external values used in an app to define UI components without hardcoding. They help maintain consistency, reduce errors, and support localization.

Types of Android Resources Used in This Practical

1. Color Resource (colors.xml)
   Stores all color codes for use across the application.
2. String Resource (strings.xml)
   Stores text messages, titles, labels, etc., making localization easier.
3. Drawable Resource (drawable/)
   Stores images (PNG, JPG, WebP) and XML shapes (rectangles, circles, gradients).
4. Dimension Resource (dimens.xml)
   Used to store sizes such as padding, margin, text size.
5. Theme/Style Resource (styles.xml)
   Defines consistent UI styles.
6. Image Resource
   Images stored under drawable folders.

1. colors.xml

File: res/values/colors.xml

```
<resources>
   <color name="appPrimary">#6200EE</color>
   <color name="appSecondary">#03DAC5</color>
   <color name="white">#FFFFFF</color>
   <color name="black">#000000</color>
</resources>
```

2. strings.xml

File: res/values/strings.xml

```
<resources>
   <string name="app_name">Resource Demo App</string>
   <string name="hello_text">Hello World using Android Resources!</string>
</resources>
```

3. dimens.xml

File: res/values/dimens.xml

```
<resources>
   <dimen name="text_large">24sp</dimen>
   <dimen name="padding_normal">20dp</dimen>
   <dimen name="image_size">150dp</dimen>
</resources>
```

4. styles.xml (Theme)

File: res/values/styles.xml

```
<resources>
   <style name="AppTheme" parent="Theme.MaterialComponents.DayNight">
      <item name="colorPrimary">@color/appPrimary</item>
      <item name="colorAccent">@color/appSecondary</item>
      <item name="android:padding">@dimen/padding_normal</item>
   </style>
</resources>
```

5. Drawable Image

Place any image (e.g., logo.png) inside:
app → res → drawable/

6. Drawable Shape (rounded_box.xml)

File: res/drawable/rounded_box.xml

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
   android:shape="rectangle">
   <solid android:color="@color/appSecondary" />
   <corners android:radius="12dp" />
</shape>
```

7. Layout File (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center"
   android:padding="@dimen/padding_normal">
   <ImageView
      android:id="@+id/imageView"
      android:layout_width="@dimen/image_size"
      android:layout_height="@dimen/image_size"
      android:src="@drawable/logo" />
   <TextView
      android:id="@+id/textView"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/hello_text"
      android:textColor="@color/appPrimary"
      android:textSize="@dimen/text_large"
      android:paddingTop="20dp" />
   <Button
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Click Me"
      android:background="@drawable/rounded_box"
```

```
        android:textColor="@color/black"
        android:padding="10dp"/>
</LinearLayout>
```

8. Java File (MainActivity.java)

```
package com.example.resourcedemo;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



3. Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

AIM: To develop an Android application that demonstrates all the Activity Lifecycle methods and analyze their execution using Logcat**.**

THEORY

An **Activity** is a single screen in an Android application. Android manages the lifecycle of an activity through a series of callback methods. These methods are automatically invoked depending on the activity's state, such as launching, pausing, stopping, restarting, or destroying.

Android Activity Lifecycle Methods

1. **onCreate()**
   - Called when the activity is created.
   - Used for initialization and loading the UI.
2. **onStart()**
   - Called when the activity becomes visible to the user.
3. **onResume()**
   - Called when the activity comes to the foreground and is ready for user interaction.
4. **onPause()**
   - Called when another activity partially covers the current activity.
5. **onStop()**
   - Called when the activity is no longer visible.
6. **onRestart()**
   - Called after the activity has been stopped and before it starts again.
7. **onDestroy()**
   - Called before the activity is destroyed.

MainActivity.java

```
package com.example.lifecycleapp;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.util.Log;
public class MainActivity extends AppCompatActivity {

   String TAG = "ActivityLifecycle";
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      Log.d(TAG, "onCreate() called");
   }

   @Override
   protected void onStart() {
      super.onStart();
      Log.d(TAG, "onStart() called");
   }

   @Override
   protected void onResume() {
      super.onResume();
      Log.d(TAG, "onResume() called");
   }
```

```
    @Override
    protected void onPause() {
        super.onPause();
        Log.d(TAG, "onPause() called");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d(TAG, "onStop() called");
    }

    @Override
    protected void onRestart() {
        super.onRestart();
        Log.d(TAG, "onRestart() called");
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        Log.d(TAG, "onDestroy() called");
    }
}
```

activity_main.xml

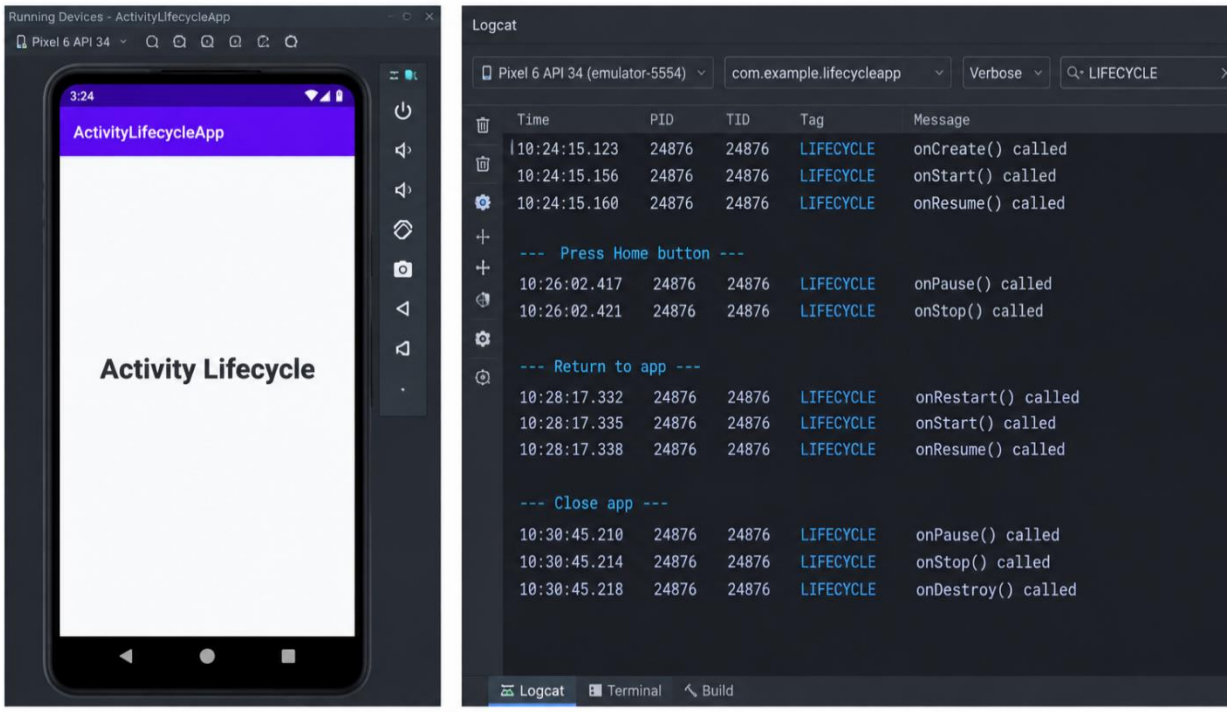
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
   <TextView
      android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Android Activity Lifecycle Demo"
        android:textSize="22sp"
        android:textStyle="bold"/>
</LinearLayout>
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.lifecycleapp">
    <application
        android:allowBackup="true"
        android:label="LifecycleApp"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
```

```
        </intent-filter>
    </activity>
</application>
</manifest>
```

Output



Practical No. 2

Fragments and Activity Interaction

1. Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.
2. Develop an Android application implementing Fragment-based UI navigation between multiple screens.
3. Develop an Android application demonstrating communication between Activity and Fragment.

AIM: Activity Life Cycle, Creating Multiple Activities, Fragment Life Cycle, and Implementing Multiple Fragments in an Android App

THEORY

1. Fragment

A Fragment is a part of user interface within an activity. It can be reused in multiple activities.

Fragment Life Cycle

- onAttach()
- onCreate()
- onCreateView()
- onActivityCreated()
- onStart()
- onResume()
- onPause()
- onStop()
- onDestroyView()
- onDestroy()
- onDetach()

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
    <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:orientation="vertical"
```

```
    android:padding="20dp">
    <Button
        android:id="@+id/btnA"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Load Fragment A"/>
    <Button
        android:id="@+id/btnB"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Load Fragment B"
        android:layout_marginTop="10dp"/>
    <FrameLayout
        android:id="@+id/fragmentContainer"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

Main Activity (MainActivity.java)

```
package com.example.fragmentsdemo;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;
```

```
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
public class MainActivity extends AppCompatActivity {
    Button btnA, btnB;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnA = findViewById(R.id.btnA);
        btnB = findViewById(R.id.btnB);
        btnA.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                loadFragment(new FragmentA());
            }
        });
        btnB.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                loadFragment(new FragmentB());
            }
        });
    }
    private void loadFragment(Fragment fragment) {
        FragmentManager fm = getSupportFragmentManager();
        FragmentTransaction ft = fm.beginTransaction();
        ft.replace(R.id.fragmentContainer, fragment);
        ft.commit();
```

```
}}
```

2. Fragment A Layout (fragment_a.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is Fragment A"
        android:textSize="24sp"/>
</LinearLayout>
```

Fragment A Java (FragmentA.java**)**

```
package com.example.fragmentsdemo;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class FragmentA extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
```

```
        return inflater.inflate(R.layout.fragment_a, container, false);
    }}
```

Fragment B Layout (fragment_b.xml**)**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is Fragment B"
        android:textSize="24sp"/>
</LinearLayout>
```

Fragment B Java (FragmentB.java)

```
package com.example.fragmentsdemo;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class FragmentB extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
```

```
                @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_b, container, false);
    }
}
```



Practical No. 3

Basic Layout Management

1. Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.
2. Develop an Android application using RelativeLayout to align UI components dynamically.
3. Develop an Android application using TableLayout to design a structured Login/Registration Form.

AIM: To Study LinearLayout, RelativeLayout, TableLayout

THEORY

Android Layouts define the structure of UI elements on the screen. Each layout arranges elements differently.

1. Linear Layout

Arranges elements vertically or horizontally.

2. Relative Layout

Positions views in relation to parent or other views.

3. Table Layout
Arranges elements in rows and columns similar to tables

1.Linear Layout

activity_linear.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">
   <TextView
       android:layout_width="wrap_content"

       android:layout_height="wrap_content"
       android:text="Linear Layout Example"
```

```
        android:textSize="22sp"
        android:padding="10dp" />
    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Button 1" />
    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Button 2"
        android:layout_marginTop="10dp" />
</LinearLayout>
```

2. Relative Layout

activity_relative.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp">
    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Relative Layout Example"
        android:textSize="22sp"/>
    <Button
        android:id="@+id/btn1"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Below Title"
        android:layout_below="@id/title"
        android:layout_marginTop="20dp"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Right of Button"
        android:layout_toRightOf="@id/btn1"
        android:layout_below="@id/title"
        android:layout_marginLeft="20dp" />
</RelativeLayout>
```

3. Table Layout

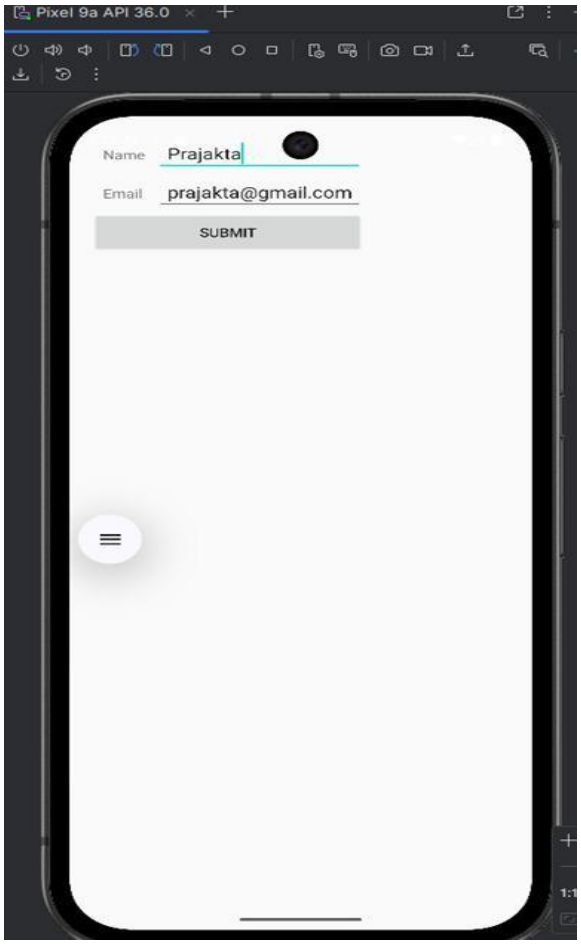
activity_table.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp">
    <TableRow>
        <TextView
            android:text="Name"
            android:padding="10dp" />
        <EditText
            android:hint="Enter name"
            android:padding="10dp"/>
    </TableRow>
    <TableRow>
```

```
        <TextView
            android:text="Email"
            android:padding="10dp" />
        <EditText
            android:hint="Enter email"
            android:padding="10dp"/>
    </TableRow>
    <TableRow>
        <Button
            android:text="Submit"
            android:layout_span="2"/>
    </TableRow>
</TableLayout>
```






Pixel 9a API 36.0
Name
Prajakta
Email
prajakta@gmail.com
SUBMIT

Practical No.4

Advanced Layout and View Handling

1. Develop an Android application using AbsoluteLayout demonstrating fixed positioning.
2. Develop an Android application using FrameLayout to overlay UI elements.
3. Develop an Android application using ScrollView to display long scrollable content.

MainActivity.java

```
package com.example.absolutelayout;

import android.app.Activity;
import android.os.Bundle;

public class MainActivity extends Activity {

   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);

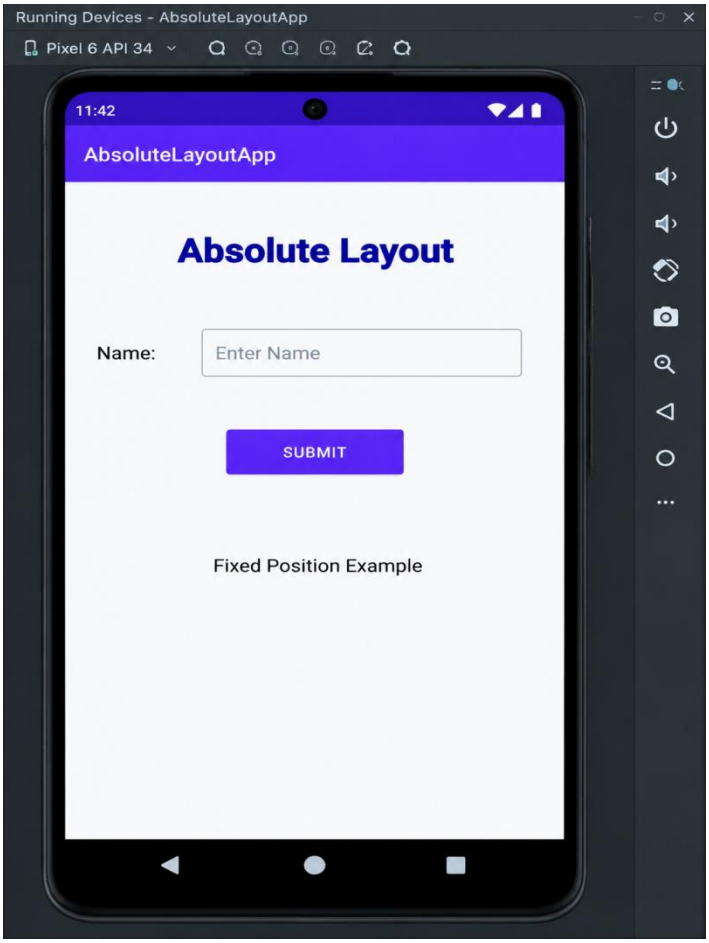
      setContentView(R.layout.activity_main);
   }
}
```

Activitty_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<AbsoluteLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent">
   <TextView
      android:id="@+id/title"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Absolute Layout"
      android:textSize="24sp"
      android:textStyle="bold"
      android:layout_x="90dp"
      android:layout_y="50dp" />
   <TextView
      android:id="@+id/nameLabel"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Name:"
      android:textSize="18sp"
      android:layout_x="30dp"
      android:layout_y="130dp" />

   <EditText
```

```
        android:id="@+id/name"
        android:layout_width="220dp"
        android:layout_height="wrap_content"
        android:hint="Enter Name"
        android:layout_x="100dp"
        android:layout_y="115dp" />
    <Button
        android:id="@+id/submit"
        android:layout_width="150dp"
        android:layout_height="wrap_content"
        android:text="SUBMIT"
        android:layout_x="120dp"
        android:layout_y="210dp" />
    <TextView
        android:id="@+id/message"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fixed Position Example"
        android:textSize="16sp"
        android:layout_x="95dp"
        android:layout_y="300dp" />
</AbsoluteLayout>
```



2. Frame Layout
activity_frame.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent">
<ImageView
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:src="@drawable/background"
   android:scaleType="centerCrop"/>
<TextView
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Frame Layout Example"
   android:textSize="22sp"
   android:layout_gravity="center"/>
</FrameLayout>
```



3. ScrollView

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:padding="16dp">
   <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:orientation="vertical">
      <TextView
         android:layout_width="match_parent"
         android:layout_height="wrap_content"
         android:text="Indian Knowledge System"
         android:textSize="28sp"
         android:textStyle="bold"
         android:gravity="center"
         android:padding="10dp"/>
      <TextView
         android:layout_width="match_parent"
         android:layout_height="wrap_content"
         android:textSize="18sp"
         android:lineSpacingExtra="8dp"
         android:text="Indian Knowledge System (IKS) is a traditional system of knowledge
developed in India over thousands of years.

It includes knowledge related to science, mathematics, medicine, astronomy, philosophy, literature,
art, architecture and education.

Mathematics in ancient India contributed concepts such as zero, decimal notation and various
mathematical methods.

Ayurveda is an ancient Indian system of medicine that focuses on maintaining health and treating
diseases through natural methods.

Indian astronomy developed detailed methods for observing celestial bodies and calculating time
and planetary movements.

Yoga is another important contribution of Indian knowledge. It promotes physical fitness,
concentration, mental well-being and discipline.

Indian Knowledge Systems are also useful for modern education because they encourage logical
thinking, problem solving, creativity and a connection between traditional knowledge and modern
technology.

Information Technology students can study IKS to understand how traditional knowledge can be
preserved, digitized and made available using modern computer technologies.
```

```
        Thus, IKS represents an important part of India's intellectual and cultural heritage." />

    </LinearLayout>

</ScrollView>
```

MainActivity.java

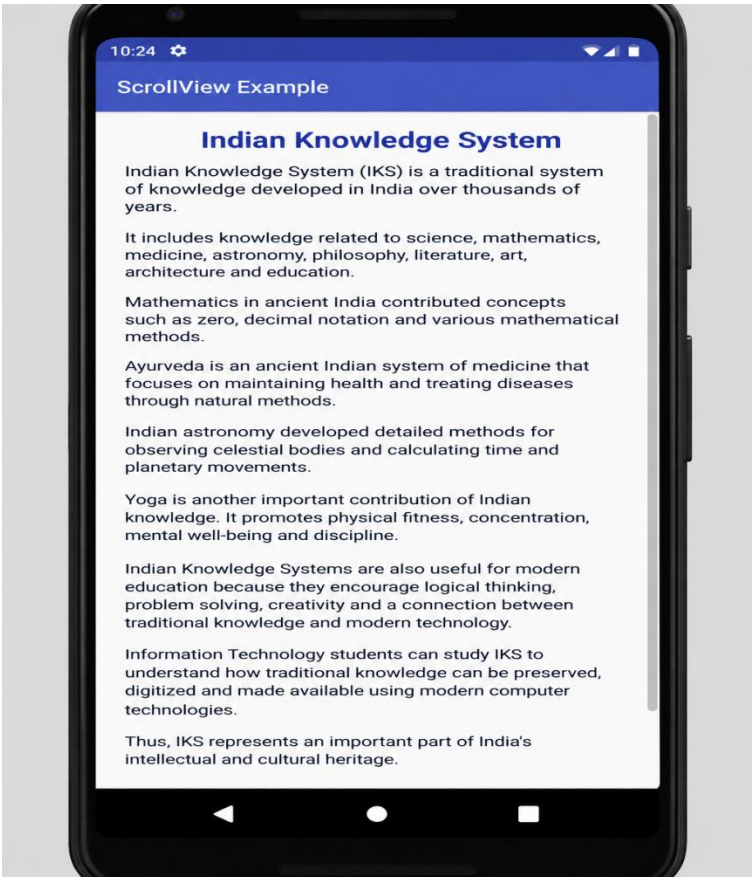
```
package com.example.scrollviewapp;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);
    }
}
```



Practical No. 5

Adapter Based Views

1. Develop an Android application using ListView with ArrayAdapter and implement item click handling.
2. Develop an Android application using GridView with Custom Adapter and image selection handling.
3. Develop an Android application implementing RecyclerView as a modern alternative to ListView.

1. ListView

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="□ Course List"
android:textSize="28sp"
android:textStyle="bold"
android:gravity="center"
android:padding="15dp" />
<ListView
android:id="@+id/listView"
android:layout_width="match_parent"
android:layout_height="match_parent" />
</LinearLayout>
```

MainActivity.java

```
package com.example.listviewdemo;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    ListView listView;

    String[] courses = {
            "B.Sc. Information Technology",
            "B.Sc. Computer Science",
            "BCA",
            "MCA",
            "M.Sc. Computer Science",
            "M.Sc. Information Technology"
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        listView = findViewById(R.id.listView);

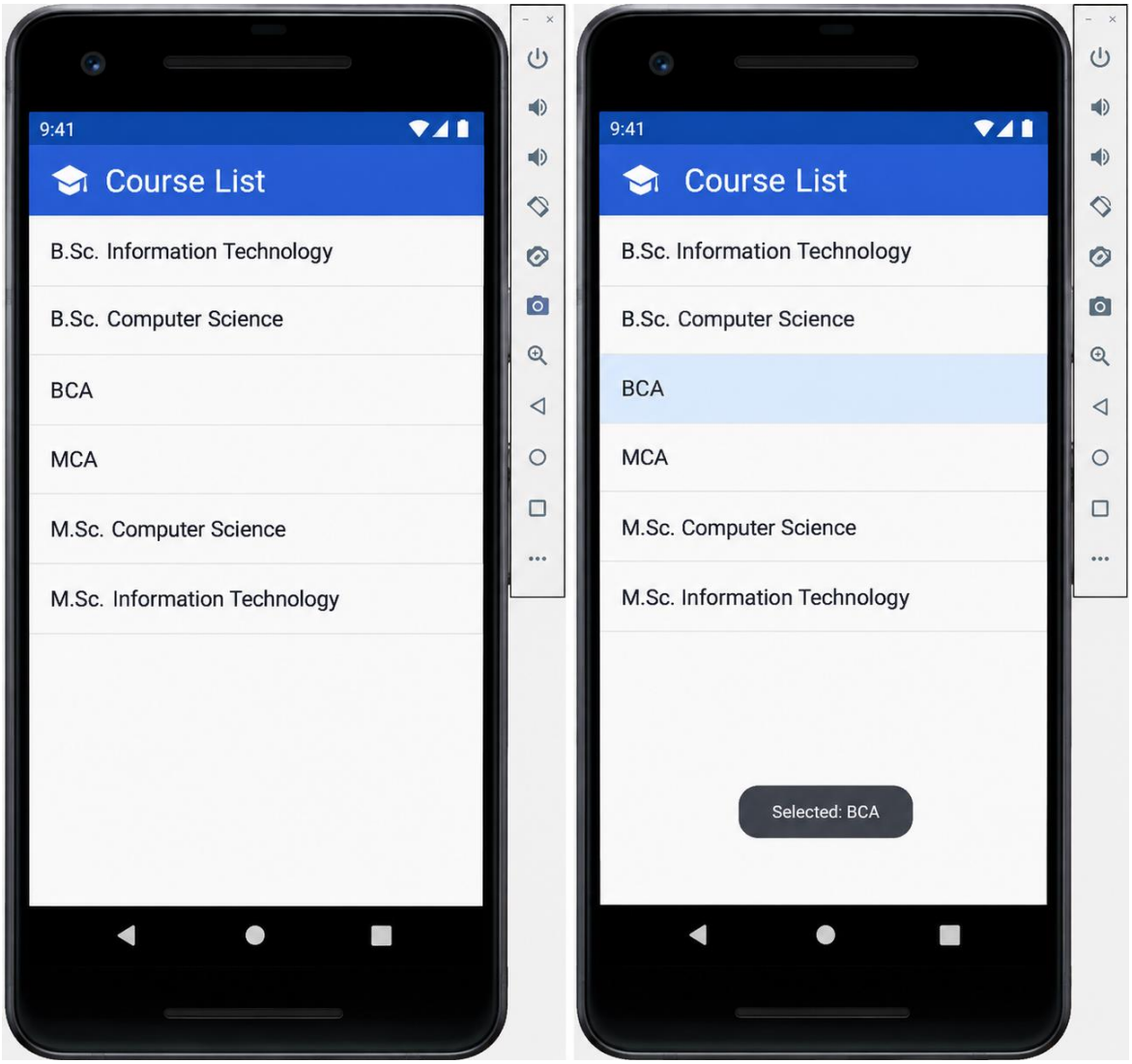
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
                this,
                android.R.layout.simple_list_item_1,
                courses
        );

        listView.setAdapter(adapter);

        listView.setOnItemClickListener((parent, view, position, id) -> {

            String selectedCourse = courses[position];

            Toast.makeText(
                    MainActivity.this,
                    "Selected: " + selectedCourse,
                    Toast.LENGTH_SHORT
            ).show();
        });
    }
}
```


9:41
Course List
B.Sc. Information Technology
B.Sc. Computer Science
BCA
MCA
M.Sc. Computer Science
M.Sc. Information Technology
9:41
Course List
B.Sc. Information Technology
B.Sc. Computer Science
BCA
MCA
M.Sc. Computer Science
M.Sc. Information Technology
Selected: BCA

2.Grid Layout

activity_grid.xml

```
<?xml version="1.0" encoding="utf-8"?>
<GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:rowCount="2"
    android:columnCount="2"
    android:padding="20dp">
    <Button
        android:text="1"
        android:layout_row="0"
        android:layout_column="0" />
    <Button
        android:text="2"
        android:layout_row="0"
        android:layout_column="1"  />
    <Button
        android:text="3"
        android:layout_row="1"
        android:layout_column="0" />
    <Button
        android:text="4"
        android:layout_row="1"
        android:layout_column="1" />
</GridLayout>
```



3.RecyclerView

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="10dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Student List"
        android:textSize="25sp"
        android:textStyle="bold"
        android:gravity="center"
        android:padding="10dp" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</LinearLayout>
```

Create item_student.xml
Right-click **res → layout → New → Layout Resource File**

item_student.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="15dp">

    <TextView
        android:id="@+id/txtName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/txtCourse"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:textSize="16sp" />

</LinearLayout>
```

Create StudentAdapter.java
Right-click your package → **New** → **Java Class**

StudentAdapter

```
package com.example.recyclerviewdemo;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;


import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

public class StudentAdapter
      extends RecyclerView.Adapter<StudentAdapter.StudentViewHolder> {

   String[] names = {
         "Amit",
         "Rahul",
         "Sneha",
         "Pooja",
         "Rohit"
   };

   String[] courses = {
         "BSc IT",
         "BSc CS",
         "BCA",
         "MCA",
         "BSc IT"
   };

   @NonNull
   @Override
   public StudentViewHolder onCreateViewHolder(
         @NonNull ViewGroup parent, int viewType) {

      View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_student, parent, false);

      return new StudentViewHolder(view);
   }

   @Override
   public void onBindViewHolder(
```

```
        @NonNull StudentViewHolder holder, int position) {

      holder.txtName.setText(names[position]);
      holder.txtCourse.setText(courses[position]);
   }

   @Override
   public int getItemCount() {
      return names.length;
   }

   static class StudentViewHolder extends RecyclerView.ViewHolder {

      TextView txtName, txtCourse;

      public StudentViewHolder(@NonNull View itemView) {
         super(itemView);

         txtName = itemView.findViewById(R.id.txtName);
         txtCourse = itemView.findViewById(R.id.txtCourse);
      }
   }
}
```

MainActivity.java

```
package com.example.recyclerviewdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
public class MainActivity extends AppCompatActivity {

   RecyclerView recyclerView;

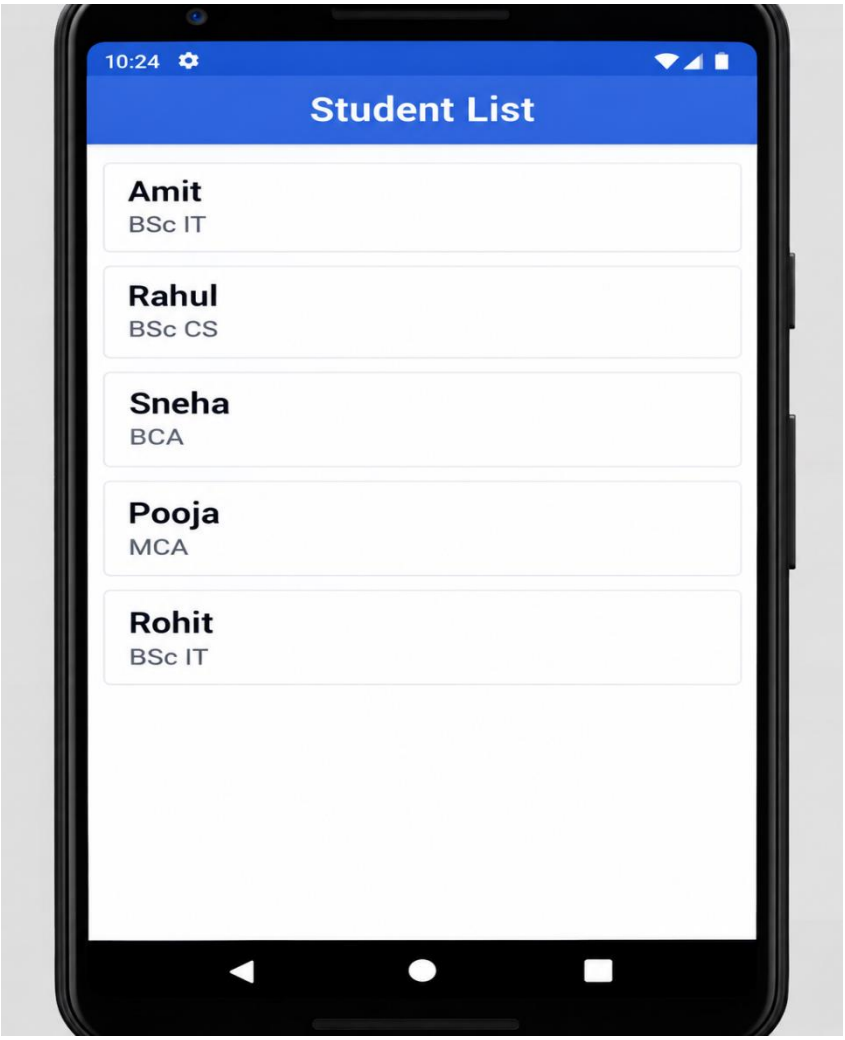
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);

      recyclerView = findViewById(R.id.recyclerView);

      recyclerView.setLayoutManager(
            new LinearLayoutManager(this));

      StudentAdapter adapter = new StudentAdapter();

      recyclerView.setAdapter(adapter);
   }
}
```


10:24
Student List
Amit
BSc IT
Rahul
BSc CS
Sneha
BCA
Pooja
MCA
Rohit
BSc IT

Module-2

Practical No 1

Menus, Dialogs and UI Interaction

1. Develop an Android application implementing AppBar (Toolbar) and Options Menu.
2. Develop an Android application demonstrating AlertDialog and custom DialogFragment.
3. Develop an Android application implementing contextual menus and popup menus.

AIM: To study and implement UI components in Android such as AppBar, TextView, EditText, Button, ImageView, Toast, and Fragment-based UI.

THEORY:Android provides a rich set of User Interface (UI) components to build interactive applications.

Common UI Components

Component	Purpose
TextView	Displays text
EditText	Accepts user input
Button	Performs an action
ImageView	Displays images
AppBar (Toolbar)	Top app bar for titles & menus
Fragment	Reusable UI section inside activity
CheckBox / RadioButton	Selection inputs
Spinner	Dropdown menu

activity_main.xml (AppBar + Fragment Container)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
    android:layout_width="match_parent"
```

```
    android:layout_height="?attr/actionBarSize"
    android:background="@color/purple_500"
        android:title="UI Components Demo"
        android:titleTextColor="@android:color/white"
        app:titleTextAppearance="@style/TextAppearance.AppCompat.Large"/>
    <FrameLayout
        android:id="@+id/frameContainer"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.uicomponents;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.FragmentTransaction;
import android.os.Bundle;
import androidx.appcompat.widget.Toolbar;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
         // Load Fragment
        FragmentTransaction ft = getSupportFragmentManager().beginTransaction();
        ft.replace(R.id.frameContainer, new HomeFragment());
        ft.commit();
    }}
```

fragment_home.xml (UI Components)

```
<?xml version="1.0" encoding="utf-8"?>
```

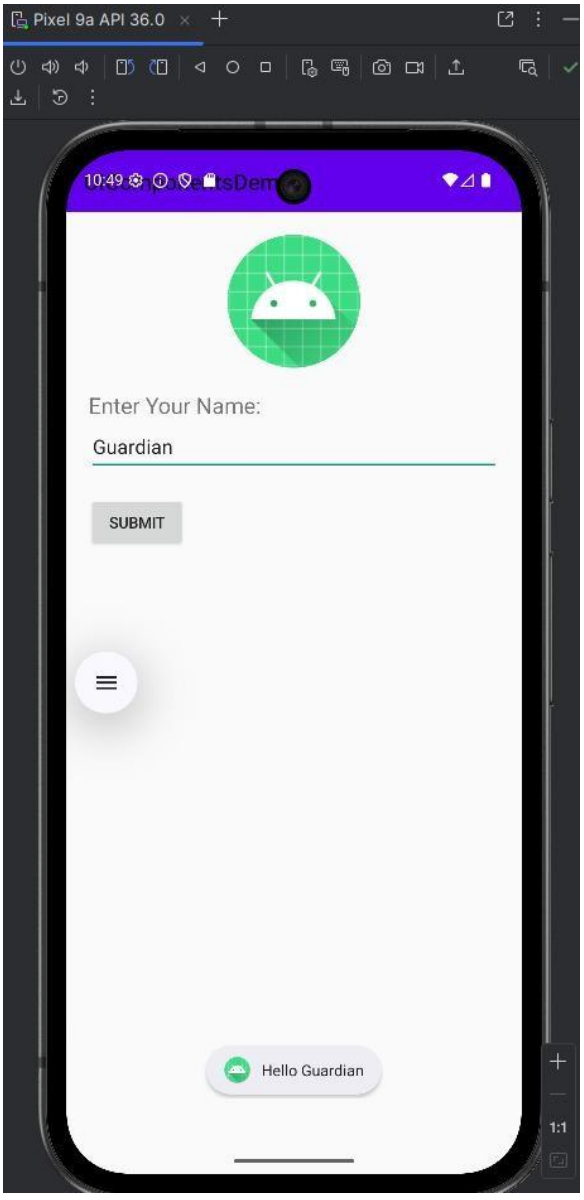
```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:orientation="vertical"
        android:padding="20dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
        <ImageView
            android:id="@+id/imgLogo"
            android:src="@drawable/ic_launcher_foreground"
            android:layout_width="120dp"
            android:layout_height="120dp"
            android:layout_gravity="center"/>
        <TextView
            android:id="@+id/txtTitle"
            android:text="Enter Your Name:"
            android:textSize="20sp"
            android:layout_marginTop="20dp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>
        <EditText
            android:id="@+id/edtName"
            android:hint="Type here"
            android:padding="10dp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <Button
            android:id="@+id/btnSubmit"
```

```
        android:text="Submit"
        android:layout_marginTop="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
  </LinearLayout>
</ScrollView>
```

HomeFragment.java

```
package com.example.uicomponents;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class HomeFragment extends Fragment {
  EditText edtName;
  Button btnSubmit;
 @Nullable
 @Override
  public View onCreateView(@NonNull LayoutInflater inflater,
   @Nullable ViewGroup container,
     @Nullable Bundle savedInstanceState) {
      View view = inflater.inflate(R.layout.fragment_home, container, false);
      edtName = view.findViewById(R.id.edtName);
      btnSubmit = view.findViewById(R.id.btnSubmit);
```

```
        btnSubmit.setOnClickListener(v -> {
            String name = edtName.getText().toString();
            Toast.makeText(getActivity(),
                    "Hello " + name,
                    Toast.LENGTH_SHORT).show();
        });
        return view;
    }}
```



AIM:To study and implement Programming menus, dialog, dialog fragments

1. Options Menu
2. Context Menu
3. Alert Dialog
4. Dialog Fragment

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

    android:orientation="vertical"

    android:gravity="center"

    android:layout_width="match_parent"

    android:layout_height="match_parent">

    <Button

        android:id="@+id/btnDialog"

        android:text="Open Alert Dialog"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"/>

    <Button

        android:id="@+id/btnFragmentDialog"

        android:text="Open Dialog Fragment"

        android:layout_marginTop="20dp"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"/>

    <TextView
```

```
    android:id="@+id/txtContext"
    android:text="Long Press Me (Context Menu)"
    android:textSize="18sp"
    android:padding="20dp"
    android:layout_marginTop="30dp"
    android:background="#DDDDDD"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
</LinearLayout>
```

Create Menu Resource

res → menu → create **menu_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
   <item
      android:id="@+id/menu_about"
      android:title="About"/>
   <item
      android:id="@+id/menu_exit"
      android:title="Exit"/>
</menu>
```

MainActivity.java

```
package com.example.menudialogdemo;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.DialogFragment;
import android.os.Bundle;
import android.view.ContextMenu;
```

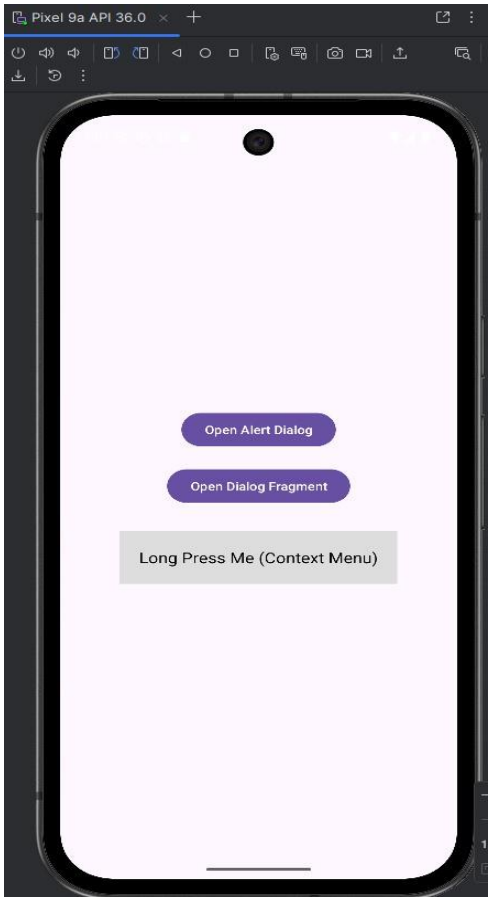
```
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
public class MainActivity extends AppCompatActivity {
Button btnDialog, btnFragmentDialog;
   TextView txtContext;
@Override
  protected void onCreate(Bundle savedInstanceState) {
       super.onCreate(savedInstanceState);
       setContentView(R.layout.activity_main);
btnDialog = findViewById(R.id.btnDialog);
        btnFragmentDialog = findViewById(R.id.btnFragmentDialog);
        txtContext = findViewById(R.id.txtContext);
// Register Context Menu
        registerForContextMenu(txtContext);
        // Alert Dialog
        btnDialog.setOnClickListener(v -> {
            AlertDialog.Builder builder = new AlertDialog.Builder(this);
            builder.setTitle("Alert")
                    .setMessage("This is Alert Dialog")
                    .setPositiveButton("OK", (dialog, which) ->
                            Toast.makeText(this, "OK Clicked", Toast.LENGTH_SHORT).show())
                    .setNegativeButton("Cancel", null)
                    .show();
 });
        // Dialog Fragment
```

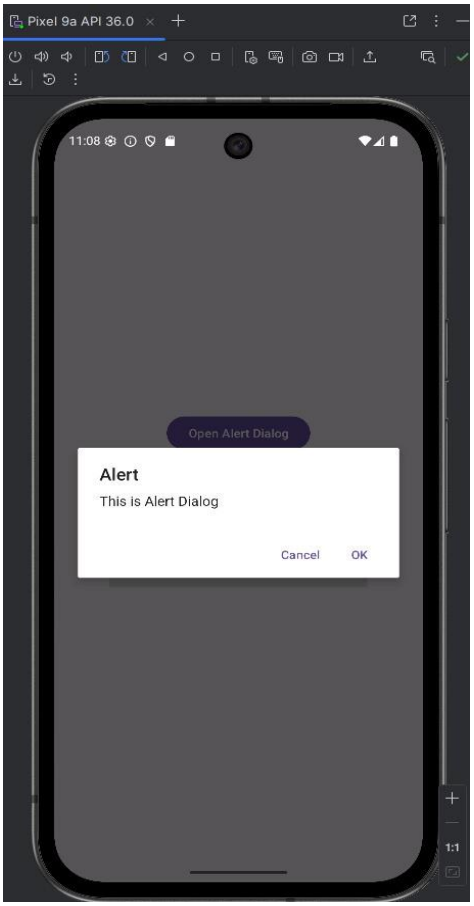
```
        btnFragmentDialog.setOnClickListener(v -> {
            DialogFragment dialog = new MyDialogFragment();
            dialog.show(getSupportFragmentManager(), "MyDialog");
        });
    }
    // Options Menu
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        if (item.getItemId() == R.id.menu_about) {
            Toast.makeText(this, "Menu: About", Toast.LENGTH_SHORT).show();
            return true;
        } else if (item.getItemId() == R.id.menu_exit) {
            finish();
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    // Context Menu
    @Override
    public void onCreateContextMenu(ContextMenu menu, View v,
                                    ContextMenu.ContextMenuInfo menuInfo) {
        super.onCreateContextMenu(menu, v, menuInfo);
        menu.setHeaderTitle("Context Menu");
        menu.add(0, 1, 0, "Edit");
```

```
        menu.add(0, 2, 0, "Delete");
    }
    @Override
    public boolean onContextItemSelected(MenuItem item) {
        if (item.getItemId() == 1) {
            Toast.makeText(this, "Edit Selected", Toast.LENGTH_SHORT).show();
        } else if (item.getItemId() == 2) {
            Toast.makeText(this, "Delete Selected", Toast.LENGTH_SHORT).show();
        }
        return true;
    }
}
```

Create Dialog Fragment Class

```
package com.example.menudialogdemo;
import android.app.Dialog;
import android.os.Bundle;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.DialogFragment;
public class MyDialogFragment extends DialogFragment {
    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        return new AlertDialog.Builder(requireActivity())
                .setTitle("Dialog Fragment")
                .setMessage("This is Dialog Fragment Example")
                .setPositiveButton("OK", null)
                .create();
    }
```


Pixel 9a API 36.0
Open Alert Dialog
Open Dialog Fragment
Long Press Me (Context Menu)


Pixel 9a API 36.0
11:08
Open Alert Dialog
Alert
This is Alert Dialog
Cancel
OK


Pixel 9a API 36.0
11:08
Open Alert Dialog
Dialog Fragment
This is Dialog Fragment Example
OK

Practical No. 2

Intents, Events and Listeners

1. Develop an Android application implementing Explicit and Implicit Intents.
2. Develop an Android application demonstrating Event Handling using Listeners
3. Develop an Android application using Adapters to populate Spinner or ListView dynamically.

AIM: To study and implement Programs on Intents, Events, Listeners and Adapters

- Explicit Intent
- Button Click Event
- Long Click Listener
- ListView with ArrayAdapter
- Toast Messages

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.example.intentdemo"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:allowBackup="true"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">
        <activity android:name=".SecondActivity"></activity>
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
```

```
android:layout_width="match_parent"

   android:layout_height="match_parent">
   <LinearLayout
      android:orientation="vertical"
      android:padding="20dp"
      android:layout_width="match_parent"
      android:layout_height="wrap_content">
      <TextView
         android:text="Intents & Events Demo"
         android:textSize="22sp"
         android:textStyle="bold"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"/>
      <!-- Intent Button -->
      <Button
         android:id="@+id/btnIntent"
         android:text="Open Second Activity"
         android:layout_marginTop="20dp"
         android:layout_width="match_parent"
         android:layout_height="wrap_content"/>
      <!-- Toast Button -->
      <Button
         android:id="@+id/btnToast"
         android:text="Show Toast Message"
         android:layout_marginTop="10dp"
         android:layout_width="match_parent"
         android:layout_height="wrap_content"/>
      <!-- Long Click Button -->
      <Button
         android:id="@+id/btnLongClick"
```

```
            android:text="Long Press Me"
            android:layout_marginTop="10dp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <!-- ListView -->
        <ListView
            android:id="@+id/listView"
            android:layout_marginTop="20dp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
    </LinearLayout>
</ScrollView>
```

MainActivity.java

```
package com.example.intentdemo;
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;
import android.widget.Toast;
public class MainActivity extends AppCompatActivity {
    Button btnIntent, btnToast, btnLongClick;
    ListView listView;
    String[] items = {"Item 1", "Item 2", "Item 3", "Item 4", "Item 5"};
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
setContentView(R.layout.activity_main);
btnIntent = findViewById(R.id.btnIntent);
btnToast = findViewById(R.id.btnToast);
btnLongClick = findViewById(R.id.btnLongClick);
listView = findViewById(R.id.listView);
// Explicit Intent
btnIntent.setOnClickListener(v -> {
   Intent intent = new Intent(MainActivity.this, SecondActivity.class);
   startActivity(intent);
});
// Click Event Listener
btnToast.setOnClickListener(v ->
       Toast.makeText(MainActivity.this,
             "Button Clicked!",
             Toast.LENGTH_SHORT).show());
// Long Click Listener
btnLongClick.setOnLongClickListener(v -> {
   Toast.makeText(MainActivity.this,
         "Long Press Detected!",
         Toast.LENGTH_SHORT).show();
   return true;
});
// Adapter Example
ArrayAdapter<String> adapter =
       new ArrayAdapter<>(this,
             android.R.layout.simple_list_item_1,
             items);

listView.setAdapter(adapter);
```

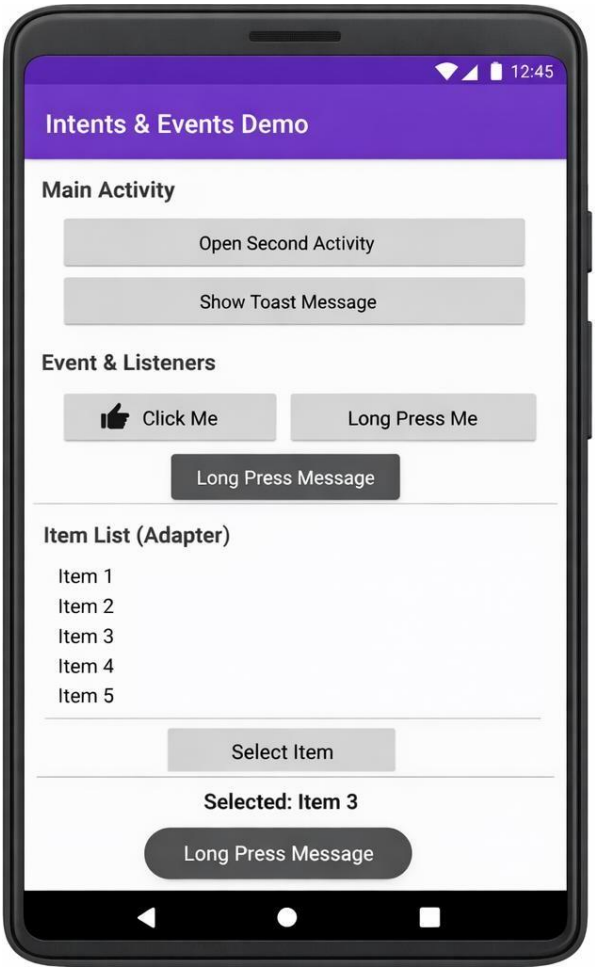
```
        // List Item Click Event
        listView.setOnItemClickListener((parent, view, position, id) ->
                Toast.makeText(MainActivity.this,
                        "Selected: " + items[position],
                        Toast.LENGTH_SHORT).show()) }`}
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:text="Welcome to Second Activity!"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

SecondActivity.java

```
package com.example.intentdemo;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
public class SecondActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
    }
}
```


12:45
Intents & Events Demo
Main Activity
Open Second Activity
Show Toast Message
Event & Listeners
Click Me
Long Press Me
Long Press Message
Item List (Adapter)
Item 1
Item 2
Item 3
Item 4
Item 5
Select Item
Selected: Item 3
Long Press Message


12:45
Welcome to Second Activity!

Practical No. 3

Services, Notifications and Broadcast Receivers

1. Develop an Android application implementing Started and Bound Services.
2. Develop an Android application implementing Notifications using Notification Manager.
3. Develop an Android application implementing Static and Dynamic Broadcast Receivers.

AIM: To study and implement Programs on Services, notification and broadcast receivers Programs on Services, notification and broadcast receivers

- Started Service
- Notification from Service
- Broadcast Receiver (BOOT_COMPLETED)
- Buttons to Start/Stop Service
- Notification Channel (Android 8+)

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.example.servicedemo"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Permission for Boot Completed -->
    <uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <application
        android:allowBackup="true"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">
        <!-- Service -->
        <service android:name=".MyService"/>
        <!-- Broadcast Receiver -->
        <receiver android:name=".BootReceiver"
            android:enabled="true"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED"/>
            </intent-filter>
```

```
        </receiver>
        <!-- Main Activity -->
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/btnStart"
        android:text="Start Service"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
    <Button
        android:id="@+id/btnStop"
        android:text="Stop Service"
        android:layout_marginTop="10dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
```

```
    <Button
        android:id="@+id/btnNotify"
        android:text="Send Notification"
        android:layout_marginTop="10dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.servicedemo;
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
public class MainActivity extends AppCompatActivity {
    Button btnStart, btnStop, btnNotify;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnStart = findViewById(R.id.btnStart);
        btnStop = findViewById(R.id.btnStop);
        btnNotify = findViewById(R.id.btnNotify);
        // Start Service
        btnStart.setOnClickListener(v -> {
            Intent intent = new Intent(this, MyService.class);
            startService(intent);
            Toast.makeText(this, "Service Started", Toast.LENGTH_SHORT).show();
        })
```

```
        // Stop Service
        btnStop.setOnClickListener(v -> {
            Intent intent = new Intent(this, MyService.class);
            stopService(intent);
            Toast.makeText(this, "Service Stopped", Toast.LENGTH_SHORT).show();
        });
        // Send Notification via Service
        btnNotify.setOnClickListener(v -> {
            Intent intent = new Intent(this, MyService.class);
            intent.setAction("SEND_NOTIFICATION");
            startService(intent);
        });
    }
}
```

MyService.java

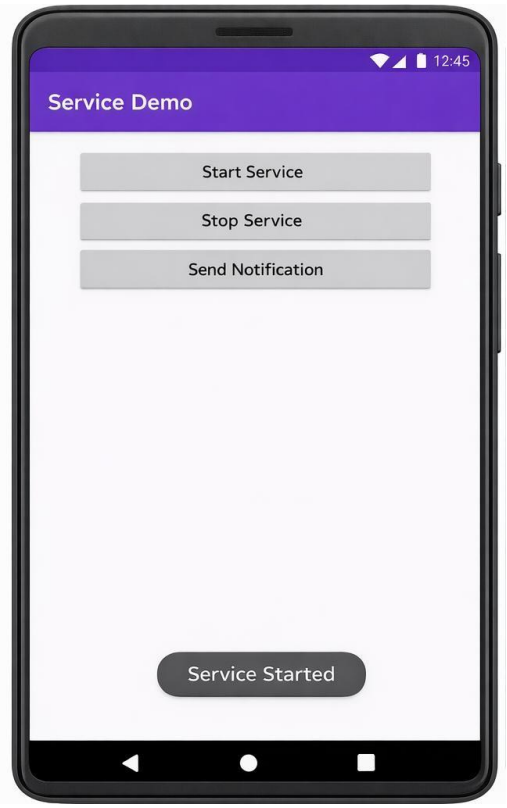
```
package com.example.servicedemo;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.Service;
import android.content.Intent;
import android.os.Build;
import android.os.IBinder;
import androidx.core.app.NotificationCompat;
public class MyService extends Service {
   private static final String CHANNEL_ID = "ServiceChannel";
  @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
if (intent != null && "SEND_NOTIFICATION".equals(intent.getAction())) {
            sendNotification()  }
```

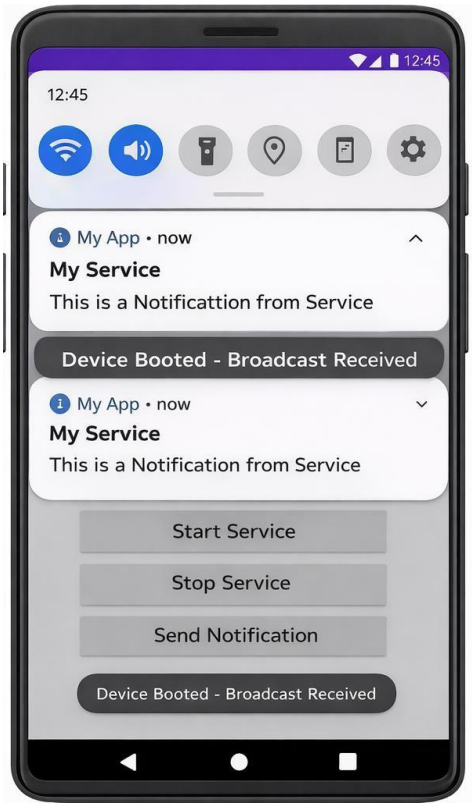
```
 return START_STICKY;
 }
private void sendNotification() {
 NotificationManager manager =
   (NotificationManager) getSystemService(NOTIFICATION_SERVICE);
// Notification Channel (For Android 8+)
     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel channel = new NotificationChannel(
              CHANNEL_ID,
              "Service Channel",
              NotificationManager.IMPORTANCE_DEFAULT);
        manager.createNotificationChannel(channel);
     }
  NotificationCompat.Builder builder =
           new NotificationCompat.Builder(this, CHANNEL_ID)
                .setContentTitle("Service Notification")
                .setContentText("This is a Notification from Service")
                .setSmallIcon(android.R.drawable.ic_dialog_info)
                .setAutoCancel(true);
 manager.notify(1, builder.build());
  }
@Override
  public IBinder onBind(Intent intent) {
     return null;
  }}
```

BootReceiver.java

```
package com.example.servicedemo;
import android.content.BroadcastReceiver;
import android.content.Context;
```

```
import android.content.Intent;
import android.widget.Toast;
public class BootReceiver extends BroadcastReceiver {
   @Override
    public void onReceive(Context context, Intent intent) {
if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())) {
     Toast.makeText(context,
           "Device Booted - Broadcast Received",
               Toast.LENGTH_LONG).show();
 // Optional: Start Service Automatically
         Intent serviceIntent = new Intent(context, MyService.class);
         context.startService(serviceIntent);
      }
   }}
```





Practical No.4

Database and Networking

1. Develop an Android application implementing SQLite database CRUD operations.
2. Develop an Android application to fetch and parse JSON data from a web service and display it in UI components.

1.Database Programming with SQLite in java language

AIM: To study and implement Database Programming with SQLite in java language

- Create SQLite Database
- Create Table
- Insert Data
- View Data
- Update Data
- Delete Data
- Display Output in TextView

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
 <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">
<EditText
            android:id="@+id/etName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter Name"/>
 <EditText
            android:id="@+id/etMarks"
            android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:hint="Enter Marks"
        android:inputType="number"/>
<Button
        android:id="@+id/btnInsert"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Insert"/>
 <Button
        android:id="@+id/btnView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="View Data"/>
    <Button
        android:id="@+id/btnUpdate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Update"/>
    <Button
        android:id="@+id/btnDelete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Delete"/>
    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="16sp"
        android:paddingTop="20dp"/>
```

```
    </LinearLayout>
</ScrollView>
```

DatabaseHelper.java

```
package com.example.sqliteexample;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
public class DatabaseHelper extends SQLiteOpenHelper {
    private static final String DATABASE_NAME = "Student.db";
    private static final String TABLE_NAME = "student_table";
    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, 1);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE_NAME +
                " (ID INTEGER PRIMARY KEY AUTOINCREMENT, NAME TEXT, MARKS
INTEGER)");
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }
    public boolean insertData(String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put("NAME", name);
        contentValues.put("MARKS", marks);
```

```
        long result = db.insert(TABLE_NAME, null, contentValues);
        return result != -1;
    }
    public Cursor getAllData() {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }
    public boolean updateData(String id, String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put("ID", id);
        contentValues.put("NAME", name);
        contentValues.put("MARKS", marks);
        db.update(TABLE_NAME, contentValues, "ID = ?", new String[]{id});
        return true;
    }
    public Integer deleteData(String id) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_NAME, "ID = ?", new String[]{id});
    }
}
```

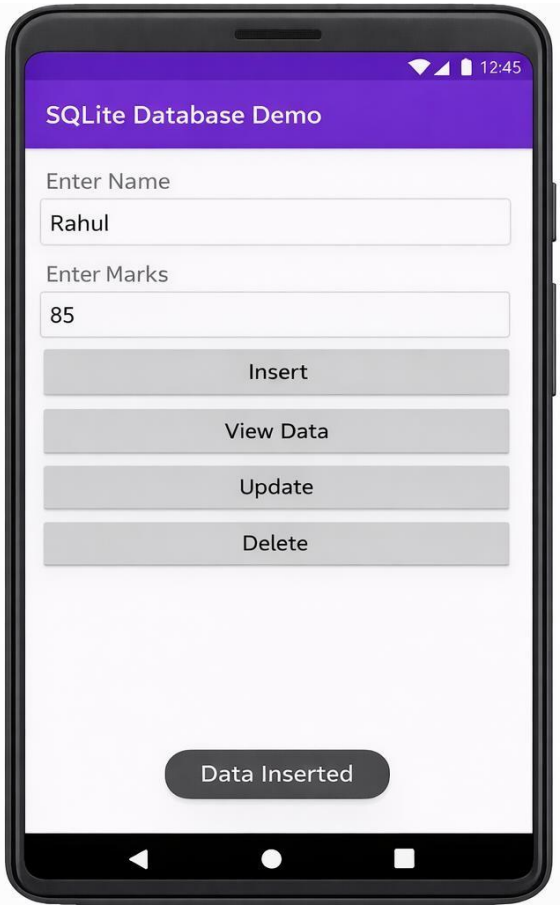
MainActivity.java

```
package com.example.sqliteexample;
import androidx.appcompat.app.AppCompatActivity;
import android.database.Cursor;
import  android.os.Bundle;
import android.view.View;
import android.widget.*;
```

```
public class MainActivity extends AppCompatActivity {
    EditText etName, etMarks;
    Button btnInsert, btnView, btnUpdate, btnDelete;
    TextView tvResult;
    DatabaseHelper myDb;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        myDb = new DatabaseHelper(this);
        etName = findViewById(R.id.etName);
        etMarks = findViewById(R.id.etMarks);
        btnInsert = findViewById(R.id.btnInsert);
        btnView = findViewById(R.id.btnView);
        btnUpdate = findViewById(R.id.btnUpdate);
        btnDelete = findViewById(R.id.btnDelete);
        tvResult = findViewById(R.id.tvResult);
        btnInsert.setOnClickListener(view -> {
            boolean isInserted = myDb.insertData(
                    etName.getText().toString(),
                    etMarks.getText().toString());
            if (isInserted)
                Toast.makeText(this, "Data Inserted", Toast.LENGTH_SHORT).show();
            else
                Toast.makeText(this, "Data Not Inserted", Toast.LENGTH_SHORT).show();
        });
        btnView.setOnClickListener(view -> {
            Cursor res = myDb.getAllData();
            if (res.getCount() == 0) {
```

```
            tvResult.setText("No Data Found");
            return;
        }
        StringBuffer buffer = new StringBuffer();
        while (res.moveToNext()) {
            buffer.append("ID : " + res.getString(0) + "\n");
            buffer.append("Name : " + res.getString(1) + "\n");
            buffer.append("Marks : " + res.getString(2) + "\n\n");
        }
        tvResult.setText(buffer.toString());
    });
    btnUpdate.setOnClickListener(view -> {
        String id = "1"; // Example: updating ID 1
        boolean isUpdate = myDb.updateData(
                id,
                etName.getText().toString(),
                etMarks.getText().toString());
        if (isUpdate)
            Toast.makeText(this, "Data Updated", Toast.LENGTH_SHORT).show();
        else
            Toast.makeText(this, "Data Not Updated", Toast.LENGTH_SHORT).show();
    });
    btnDelete.setOnClickListener(view -> {
        String id = "1"; // Example: deleting ID 1
        Integer deletedRows = myDb.deleteData(id);
        if (deletedRows > 0)
            Toast.makeText(this, "Data Deleted", Toast.LENGTH_SHORT).show();
        else
            Toast.makeText(this, "Data Not Deleted", Toast.LENGTH_SHORT).show();
```

```
        });
    }
}
```


12:45
SQLite Database Demo
Enter Name
Rahul
Enter Marks
85
Insert
View Data
Update
Delete
Data Inserted


12:45
SQLite Database Demo
Insert
View Data
Update
Delete
ID : 1
Name : Rahul
Marks : 85

2.Programming Network Communications and Services (JSON)

AIM:To study and implement Programming Network Communications and Services (JSON)

- Connect to Internet
- Fetch JSON data from server
- Parse JSON
- Display data in TextView

We will fetch JSON from:
https://jsonplaceholder.typicode.com/users

Add Internet Permission

Open **AndroidManifest.xml**

Inside <manifest> tag.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">
        <Button
            android:id="@+id/btnFetch"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Fetch JSON Data"/>
```

```
        <TextView
            android:id="@+id/tvResult"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="16sp"
            android:paddingTop="20dp"/>
    </LinearLayout>
</ScrollView>
```

MainActivity.java

```
package com.example.jsonexample;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import org.json.JSONArray;
import org.json.JSONObject;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
public class MainActivity extends AppCompatActivity {
    Button btnFetch;
    TextView tvResult;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnFetch = findViewById(R.id.btnFetch);
        tvResult = findViewById(R.id.tvResult);
```

```
        btnFetch.setOnClickListener(v -> fetchJSON());
    }
    private void fetchJSON() {
        new Thread(() -> {
            try {
                URL url = new URL("https://jsonplaceholder.typicode.com/users");
                HttpURLConnection conn = (HttpURLConnection) url.openConnection();
                conn.setRequestMethod("GET");
                BufferedReader reader = new BufferedReader(
                        new InputStreamReader(conn.getInputStream()));
                StringBuilder result = new StringBuilder();
                String line;
                while ((line = reader.readLine()) != null) {
                    result.append(line);
                }
                JSONArray jsonArray = new JSONArray(result.toString());
                StringBuilder displayData = new StringBuilder();
                for (int i = 0; i < jsonArray.length(); i++) {
                    JSONObject obj = jsonArray.getJSONObject(i);
                    displayData.append("Name: ")
                            .append(obj.getString("name")).append("\n");
                    displayData.append("Email: ")
                            .append(obj.getString("email")).append("\n\n");
                }
                runOnUiThread(() ->
                        tvResult.setText(displayData.toString()));
            } catch (Exception e) {
                e.printStackTrace();
            }
```

```
}
).start();
   }}
```


11:04 AM
92%
jsonexample
[Fetch JSON Data]
Name: Leanne Graham
Email: Sincere@april.biz
Name: Ervin Howell
Email: Shanna@melissa.tv
Name: Clementine Bauch
Email: Nathan@yesenia.net
Name: Patricia Lebsack
Email: Karianne@noemi.tv

Practical No. 5

Concurrency, Media and Security

1. Develop an Android application demonstrating Threads and Handlers for background processing.
2. Develop an Android application implementing Media APIs and Telephone APIs.
3. Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml.

1. Develop an Android application demonstrating Threads and Handlers for background processing.

MainActivity.java

```
package com.example.myapplication;
import android.annotation.SuppressLint;
import android.os.Bundle;
import android.os.Handler;
import android.os.Message;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
   TextView textView;
   Handler handler;
   @SuppressLint("MissingInflatedId")
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      textView = findViewById(R.id.textView);
      handler = new Handler() {
         @Override
         public void handleMessage(Message msg) {
            textView.setText("Count : " + msg.arg1);
         }
      };
      Thread thread = new Thread(new Runnable() {
         @Override
         public void run() {
            for(int i=1;i<=10;i++){
               Message message = new Message();
               message.arg1 = i;
               handler.sendMessage(message);
               try {
                  Thread.sleep(1000);
```

```
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }


    });
    thread.start();  }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:gravity="center"
   android:orientation="vertical">
   <TextView
      android:id="@+id/textView"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Thread Example"
      android:textSize="25sp"/>
</LinearLayout>
```


My Application
Count : 10

2.Programming media API and telephone API

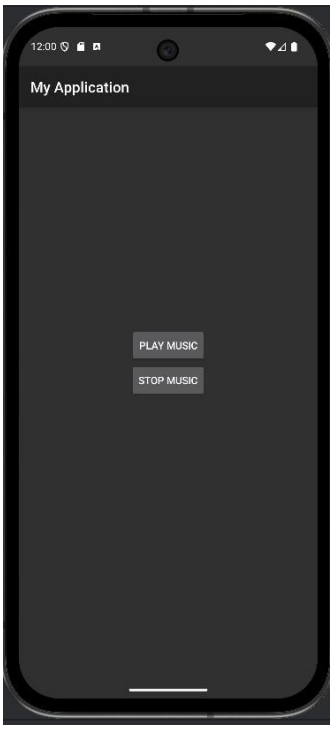
activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">
    <Button
        android:id="@+id/play"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Play Music"/>
    <Button
        android:id="@+id/stop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop Music"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.mediaapi;
import android.media.MediaPlayer;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Button play, stop;
```

```
    MediaPlayer mediaPlayer;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        play = findViewById(R.id.play);
        stop = findViewById(R.id.stop);
        mediaPlayer = MediaPlayer.create(this, R.raw.music);
        play.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                mediaPlayer.start();
            }
        });
        stop.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                mediaPlayer.stop();
            }
        });
    }
}
```



activity_main.xml

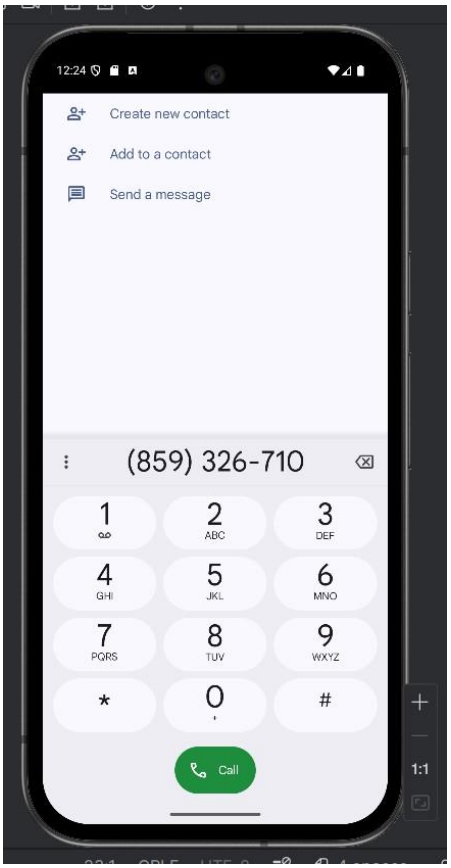
```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">
    <EditText
        android:id="@+id/number"
        android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:hint="Enter Phone Number"/>
    <Button
        android:id="@+id/call"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Call"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.telephoneapi;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    EditText number;
    Button call;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    number = findViewById(R.id.number);
    call = findViewById(R.id.call);
    call.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            String num = number.getText().toString();
            Intent intent = new Intent(Intent.ACTION_DIAL);
            intent.setData(Uri.parse("tel:" + num));
            startActivity(intent);
        } });   } }
```





3.Programming security and permissions

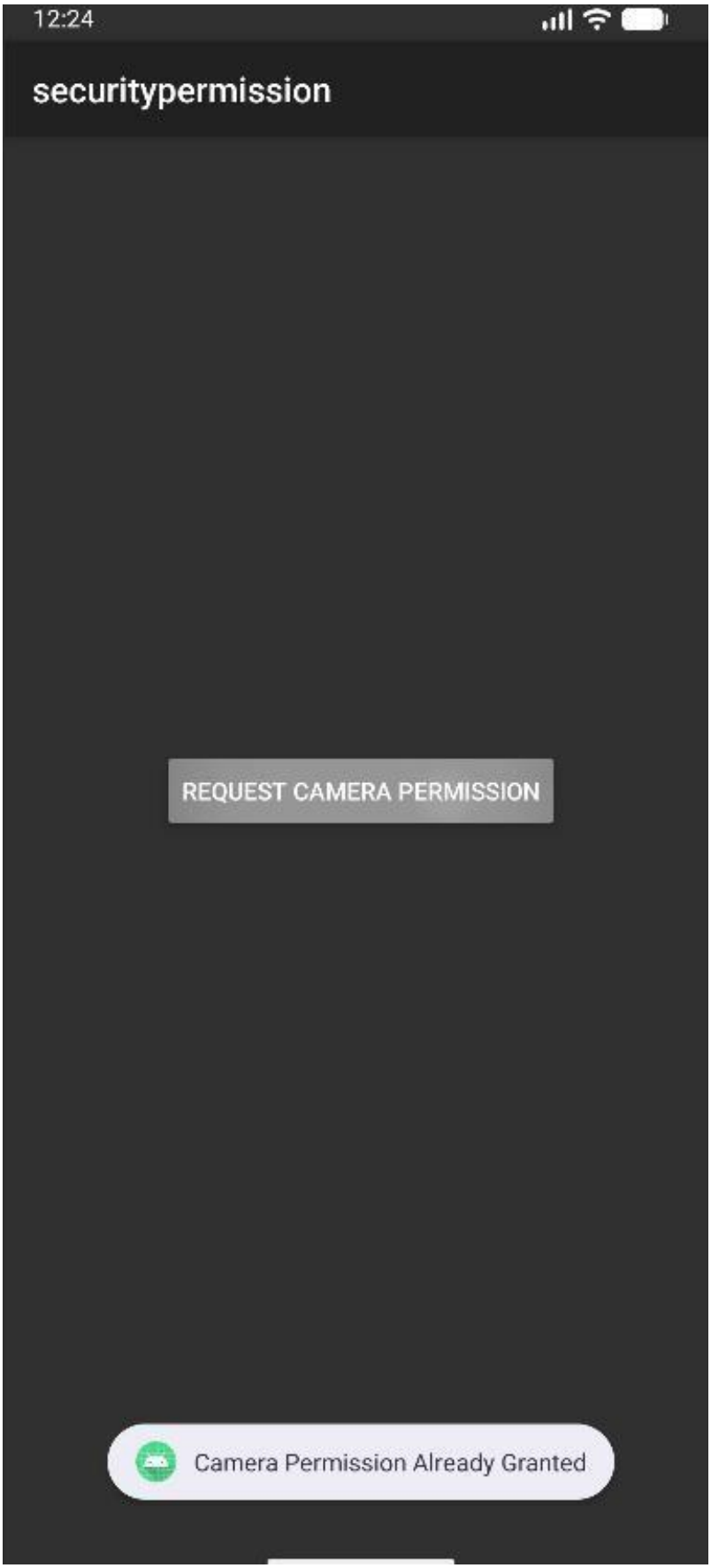
activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
    <Button
        android:id="@+id/btnPermission"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Request Camera Permission"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.securitypermission;
import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
public class MainActivity extends AppCompatActivity {
    Button btnPermission;
    private static final int CAMERA_PERMISSION_CODE = 100;
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnPermission = findViewById(R.id.btnPermission);
        btnPermission.setOnClickListener(v -> checkPermission());
    }
    private void checkPermission() {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
                == PackageManager.PERMISSION_GRANTED) {
            Toast.makeText(this, "Camera Permission Already Granted",
Toast.LENGTH_SHORT).show();
        } else {
            ActivityCompat.requestPermissions(this,
                    new String[]{Manifest.permission.CAMERA},
                    CAMERA_PERMISSION_CODE);
        }   }
    @Override
    public void onRequestPermissionsResult(int requestCode, String[] permissions, int[]
grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == CAMERA_PERMISSION_CODE) {
            if (grantResults.length > 0 && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
                Toast.makeText(this, "Camera Permission Granted",
Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this, "Camera Permission Denied",
Toast.LENGTH_SHORT).show();
            }
    }    } }
```


12:24
securitypermission
REQUEST CAMERA PERMISSION
Camera Permission Already Granted